

Product Analyst Intern: Assignment

by Shriya Konduru

Task 1: What doesn't match

1. Total does not equal subtotal + tax + shipping
2. Decimal amounts instead of integer cents
3. Undocumented status value
4. `customer.email` is not always present
5. `has_more` contradicts `next_cursor`
6. Missing order returns 200 instead of 404

Worst finding: the wrong HTTP status (#6)

Task 2: Total revenue: \$230.70

Task 3: Email to Priya

Task 4: Bug report

Task 1: What doesn't match

1. Total does not equal subtotal + tax + shipping

Docs claim: "total always equals subtotal + tax + shipping"

Data: `ord_1004` (orders_page1.json)

Field	Value
subtotal	6200
tax	511
shipping	599
subtotal + tax + shipping	7310
<code>total</code> field	6810

- Subtotal is independently verifiable (unit price × quantity); total has no such check. That's the only reason total is treated as broken here

- Impact: billing or revenue reconciliation off this field gets a silently wrong number

2. Decimal amounts instead of integer cents

Docs claim: "all monetary amounts are integers in the smallest unit of the currency," e.g. "\$54.70 is returned as 5470"

Data: `ord_1006` (orders_page2.json): subtotal 44.0, tax 3.63, shipping 5.99, total 53.62

- Internally consistent ($44.0 + 3.63 + 5.99 = 53.62$), but breaks the documented format outright
- Impact: silently corrupts any cross-order total unless normalized

3. Undocumented status value

Docs claim: status is one of pending / shipped / delivered / cancelled

Data: `ord_1003` (orders_page1.json) has status `"refunded"`, outside the documented set

- Impact: status-branching code hits an unhandled case

4. `customer.email` is not always present

Docs claim: "the customer's email address. Always present."

Data: `ord_1005` (orders_page2.json) is a guest order with `email: null`

- Impact: anything emailing customers off this field fails or silently skips it

5. `has_more` contradicts `next_cursor`

Docs claim: check `has_more` to know if another page exists

Data: `orders_page1.json` returns `has_more: false` with a non-null `next_cursor`, which fetched `orders_page2.json`, holding two further real orders

- Impact: a caller trusting `has_more` silently loses a third of the data

6. Missing order returns 200 instead of 404

Docs claim: `GET /v1/orders/{id}` "returns 404 if no order with that ID exists"

Data: `order_ord_9999.json`, captured from exactly that request, came back **200** with `{"order": null}`

Worst finding: the wrong HTTP status (#6)

- `ord_9999` doesn't exist. Docs say this should return 404; it actually returned 200 with a null order.
- Every other finding produces a value visible on inspecting the data. This one breaks the mechanism callers use to trust a response at all: status codes exist so bodies don't need manual inspection.
- Code checking `status == 404` never triggers here, since status is 200. It treats the null order as success and either crashes or proceeds with invalid data.

Task 2: Total revenue: \$230.70

- Used both pages, since page 1's `has_more: false` is contradicted by page 2 existing (#5)
- Converted `ord_1006` to cents ($\times 100$), per the docs' own rule, rather than dropping it
- Used subtotal + tax + shipping (7310) over `total` (6810) for `ord_1004`, same reasoning as #1
- Excluded `ord_1003` (refunded); refunded money isn't revenue, though this status is undocumented (#3)

Sum: $5470 + 2381 + 7310 + 2547 + 5362 = 23,070$ cents = **\$230.70**

Task 3: Email to Priya

Hi Priya,

Thanks for flagging this. Two separate issues turned up.

First, the field indicating more order pages exist was showing false when more existed, so a pull relying on it would have missed real orders.

Second, one order's amount was recorded in dollars instead of cents, undercounting it in the total.

Correcting both, the numbers reconcile. Both are logged as bugs on our end, so happy to walk through the figures together if useful.

Best Regards,
Shriya Konduru

Task 4: Bug report

Bug: `GET /v1/orders/{id}` returns 200 instead of 404 for a nonexistent order

What to look at:

The order-lookup endpoint, `GET /v1/orders/{id}`, specifically the path handling a missing/nonexistent order ID.

What happens:

Requesting an order ID that doesn't exist (e.g. `ord_9999`) returns HTTP status 200, with the response body `{"order": null}`. Per the documentation, this case is supposed to return a 404.

What should happen instead:

The endpoint should return HTTP status 404 when no order matches the given ID, consistent with the documented behavior, instead of a 200 with a null payload.

Why this matters:

Callers conventionally rely on the HTTP status code to distinguish success from failure, rather than inspecting the response body on every call. Any client that checks `if status == 404` to handle a missing order will never catch this case, since the status is 200. It will instead treat the response as successful and attempt to use `order.total`, `order.customer`, etc. on a null value, causing a crash or a silently invalid state, depending on how defensively the calling code was written.

Suggested fix:

Update the handler for `GET /v1/orders/{id}` to return a 404 status when the lookup finds no matching order, instead of a 200 with a null `order` field. Add a regression test asserting this specific case (lookup on a known-invalid ID returns 404, not 200).